Laboratório Prolog · Aula 13

Heitor Freitas
Gerado em 20/10/2026, 09:14:34 · tempo registrado no cronômetro: 0:00

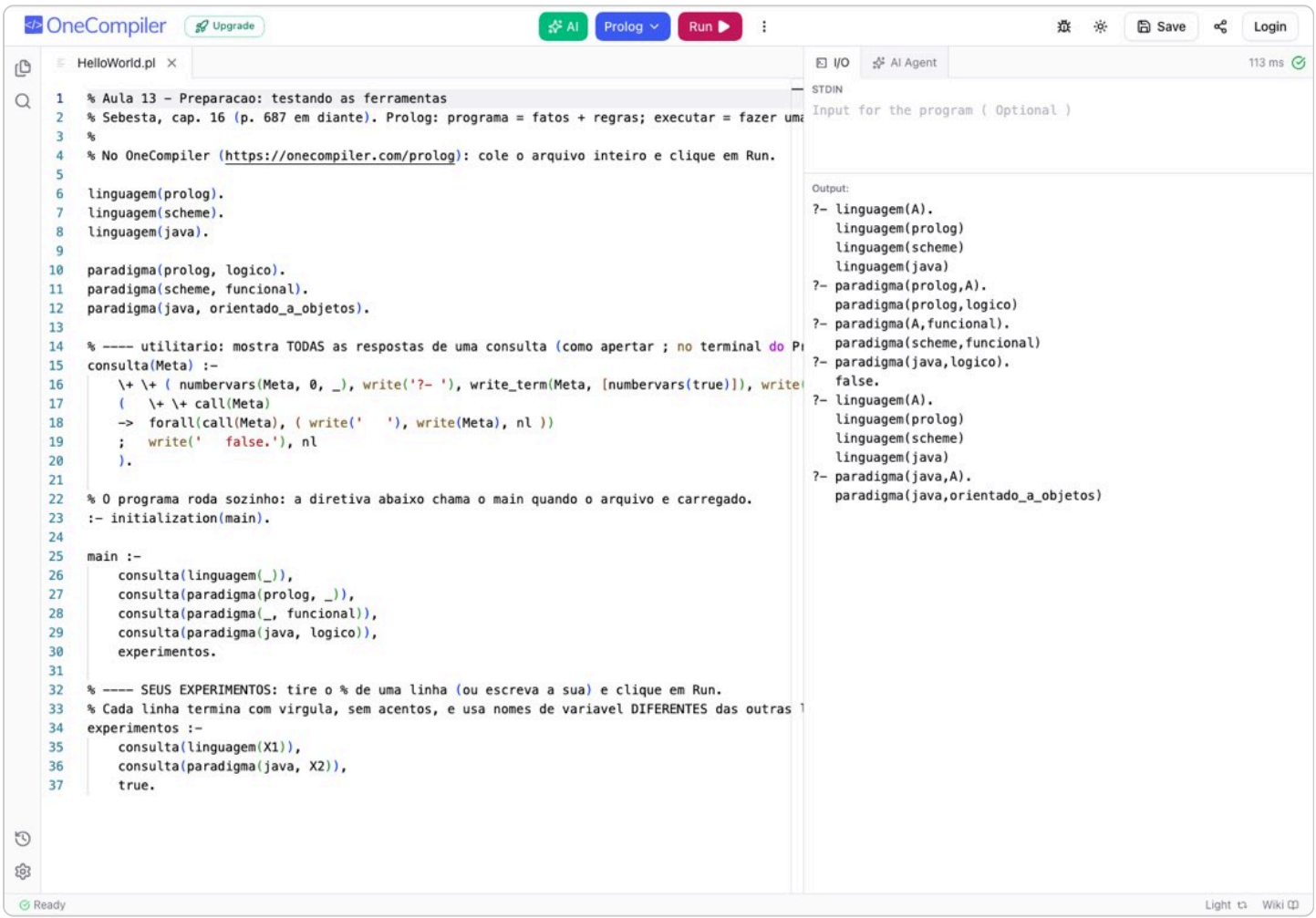
Preparação: Programar descrevendo, não instruindo

Previsão: O que paradigma(L, funcional) e paradigma(java, logico) vão responder?

paradigma(L, funcional) deve responder L = scheme, porque é o único fato com funcional. paradigma(java, logico) deve dar false, porque esse fato não está na base.

Saída do ola.pl

```
Saída no print do OneCompiler em anexo (código + saída).
```



0.1 Qual é a diferença entre paradigma(prolog, P) e paradigma(prolog, logico)? O que cada uma pergunta?

paradigma(prolog, P) tem uma variável: pergunta "qual é o paradigma do prolog?" e o Prolog procura o valor de P que torna a consulta verdadeira (respondeu logico). paradigma(prolog, logico) não tem variável: é uma pergunta de sim ou não, "prolog é do paradigma lógico?", e a resposta só pode ser true ou false.

0.2 paradigma(java, logico) deu false: Java não é lógico ou o programa não sabe isso? Justifique.

Quer dizer que "o programa não sabe isso", ou melhor, não consegue provar. O Prolog só responde com base nos fatos e regras da base: como não existe o fato paradigma(java, logico) nem regra que o deduza, ele responde false

(hipótese do mundo fechado). O false não é uma afirmação sobre o mundo real, é "não foi possível provar com o que eu sei".

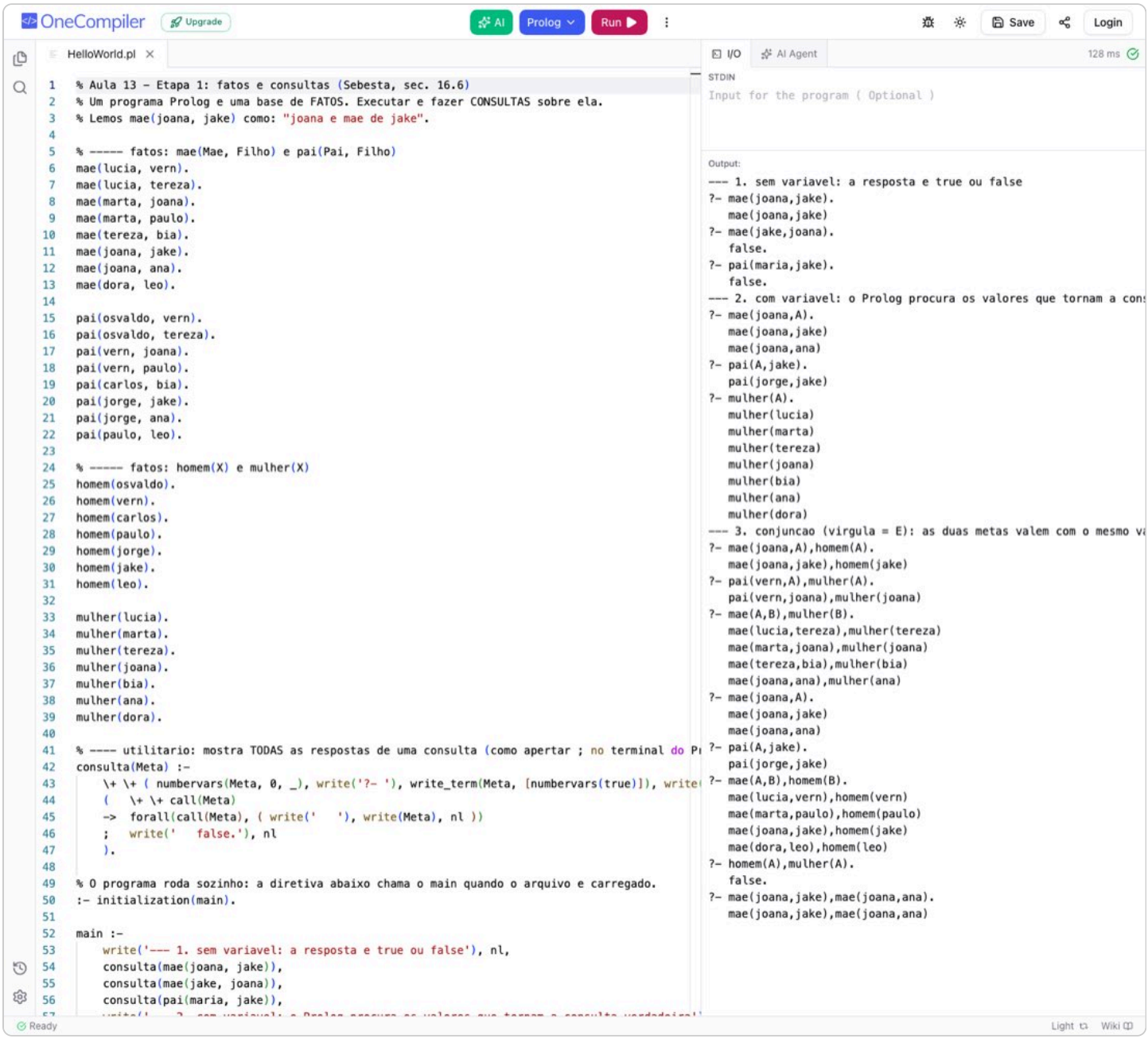
Etapa 1: Uma base de fatos e perguntas sobre ela

Previsão: Quantas respostas terá mulher(Quem)? E quais pares terá (mae(M, F), mulher(F))?

mulher(Quem) deve ter 7 respostas (lucia, marta, tereza, joana, bia, ana, dora). (mae(M, F), mulher(F)) deve dar as mães com filhas: lucia-tereza, marta-joana, tereza-bia e joana-ana.

Saída do familia.pl

```
Saída no print do OneCompiler em anexo (código + saída).
```



1.1 Escreva em português o que significa a consulta (mae(Mae, Filha), mulher(Filha)) e quantas respostas ela teve.

Significa: "quais são os pares mãe e filha, ou seja, Mae é mãe de Filha e Filha é mulher?". A vírgula é o E, e a variável Filha precisa ter o mesmo valor nas duas metas. Teve 4 respostas: lucia/tereza, marta/joana, tereza/bia e joana/ana.

1.2 pai(maria, jake) deu false mas a base não menciona maria. Que pressuposto explica? Por que pode ser um problema?

A hipótese do mundo fechado: tudo o que não está na base e não pode ser provado é considerado falso. Na prática isso pode ser um problema porque a base quase nunca está completa: um sistema que diz false para algo que ele simplesmente não conhece pode levar a conclusões erradas (por exemplo, dizer que um paciente não tem uma alergia só porque ela não foi cadastrada). "Não sei" e "é falso" viram a mesma resposta.

1.3 Em que ordem as respostas de mulher(Quem) apareceram? O que determina essa ordem?

Apareceram na ordem lucia, marta, tereza, joana, bia, ana, dora, que é exatamente a ordem em que os fatos mulher/1 foram escritos no arquivo. O Prolog percorre as cláusulas de cima para baixo e, a cada backtracking, tenta o próximo fato que unifica, então a ordem das respostas é a ordem do código.

Etapa 2: Regras: ensinando o Prolog a deduzir

Previsão: Quais respostas terão tio(X, jake) e primo(leo, X)?

tio(X, jake): paulo (irmão de joana, mãe de jake). primo(leo, X): jake e ana (leo é filho de paulo, e paulo é irmão de joana, mãe de jake e ana).

Previsão: Quais unificações vão falhar: f(X,X)=f(a,b), f(X,X)=f(a,Y), [A|B]=[], 2+3=5?

Vão falhar f(X, X) = f(a, b) (X não pode ser a e b ao mesmo tempo), [A|B] = [] (a lista vazia não tem cabeça) e 2 + 3 = 5 (= não calcula, compara a estrutura +(2,3) com o número 5). f(X, X) = f(a, Y) unifica com X = a e Y = a.

Saídas de regras.pl e unificacao.pl

```
Saída no print do OneCompiler em anexo (código + saída).
```

OneCompiler Upgrade AI Prolog Run Save Login

HelloWorld.pl

```
% Aula 13 - Etapa 2: regras e recursao (Sebesta, sec. 16.6)
% Uma REGRA tem a forma  cabeca :- corpo.   Le-se: "a cabeca e verdadeira SE o corpo for verda
% A virgula no corpo significa E. Variaveis comecam com letra MAIUSCULA.
% Duas regras com a mesma cabeca significam OU: vale qualquer uma delas.

% ----- fatos: mae(Mae, Filho) e pai(Pai, Filho)
mae(lucia, vern).
mae(lucia, tereza).
mae(marta, joana).
mae(marta, paulo).
mae(tereza, bia).
mae(joana, jake).
mae(joana, ana).
mae(dora, leo).

pai(osvaldo, vern).
pai(osvaldo, tereza).
pai(vern, joana).
pai(vern, paulo).
pai(carlos, bia).
pai(jorge, jake).
pai(jorge, ana).
pai(paulo, leo).

% ----- fatos: homem(X) e mulher(X)
homem(osvaldo).
homem(vern).
homem(carlos).
homem(paulo).
homem(jorge).
homem(jake).
homem(leo).

mulher(lucia).
mulher(marta).
mulher(tereza).
mulher(joana).
mulher(bia).
mulher(ana).
mulher(dora).

% ----- regras
progenitor(X, Y) :- mae(X, Y).          % X e progenitor de Y se X e mae de Y
progenitor(X, Y) :- pai(X, Y).          % ... OU se X e pai de Y

avo(X, Z) :- progenitor(X, Y), progenitor(Y, Z).

irmao(X, Y) :- mae(M, X), mae(M, Y), pai(P, X), pai(P, Y), X \== Y.

tio(X, Y) :- progenitor(P, Y), irmao(X, P), homem(X).

primo(X, Y) :- progenitor(P, X), progenitor(Q, Y), irmao(P, Q).

% ----- regra RECURSIVA: ancestral
ancestral(X, Y) :- progenitor(X, Y).                    % caso base
ancestral(X, Y) :- progenitor(X, Z), ancestral(Z, Y).   % caso recursivo

% ---- utilitario: mostra TODAS as respostas de uma consulta (como apertar ; no terminal do Pr
consulta(Meta) :-
    \+ \+ ( numbervars(Meta, 0, _), write('?- '), write_term(Meta, [numbervars(true)]), write(
    (   \+ \+ call(Meta)
    ->  forall(call(Meta), ( write('   '), write(Meta), nl ))
    ;   write('   false.'), nl
    ).

% O programa roda sozinho: a diretiva abaixo chama o main quando o arquivo e carregado.
:- initialization(main).

main :-
    write('--- avo: duas regras progenitor encadeadas'), nl,
    consulta(avo(_, jake)),
    consulta(avo(osvaldo, _)),
    write('--- irmao, tio e primo'), nl,
    consulta(irmao(jake, _)),
    consulta(tio(_, jake)),
    consulta(primo(leo, _)),
    write('--- ancestral (recursiva)'), nl,
    consulta(ancestral(osvaldo, _)),
    consulta(ancestral(_, jake)),
    consulta(ancestral(jake, _)),
    experimentos.

% ---- SEUS EXPERIMENTOS: tire o % de uma linha (ou escreva a sua) e clique em Run.
% Cada linha termina com virgula, sem acentos, e usa nomes de variavel DIFERENTES das outras 
experimentos :-
    consulta(irmao(X1, Y1)),
    consulta(tio(X2, Y2)),
    consulta(ancestral(X3, leo)),
    consulta(primo(X4, Y4)),
    findall(X5, ancestral(osvaldo, X5), L5), write(L5), nl,
    true.
```

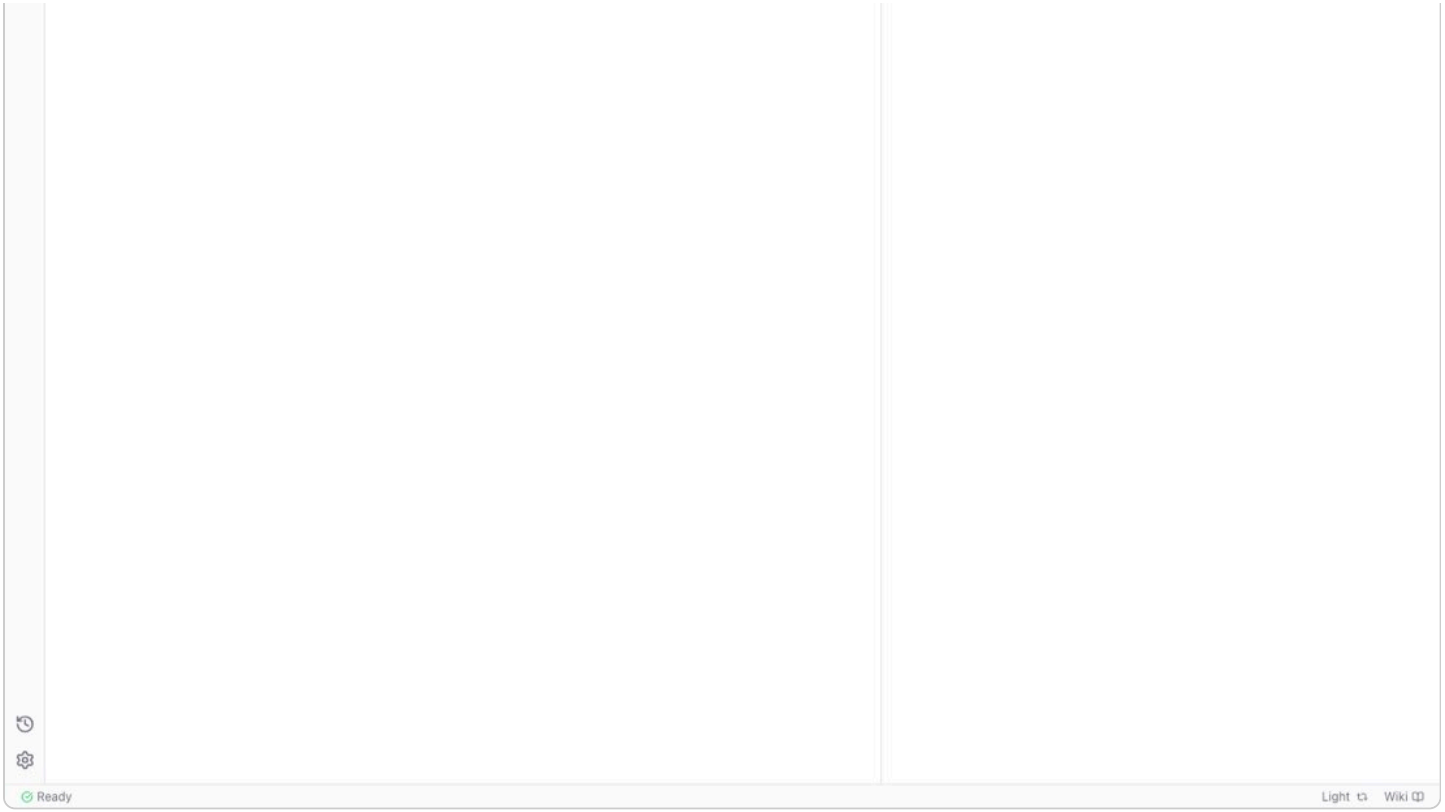
I/O AI Agent 127 ms

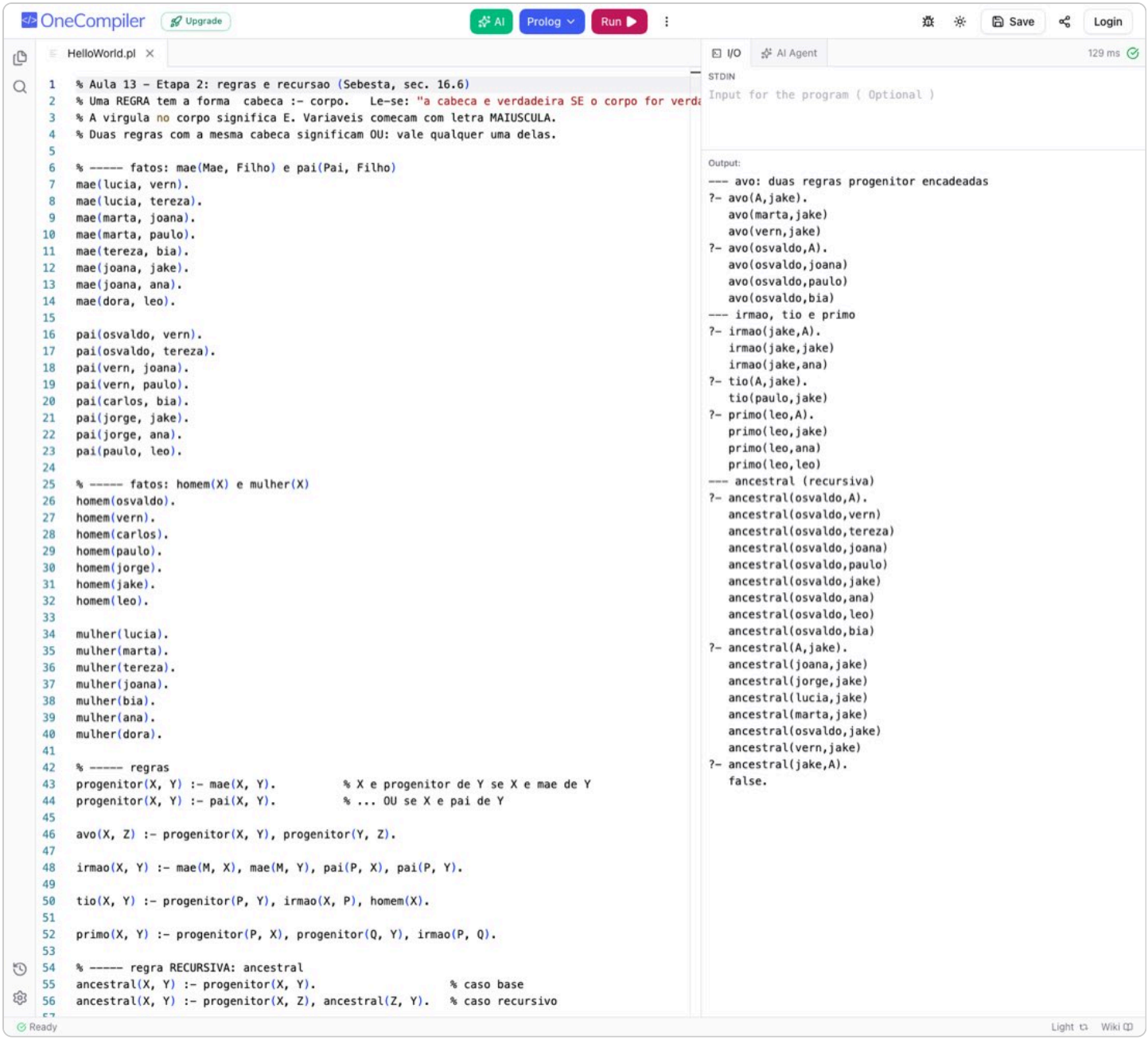
STDIN

Input for the program (Optional)

Output:

```
--- avo: duas regras progenitor encadeadas
?- avo(A,jake).
   avo(marta,jake)
   avo(vern,jake)
?- avo(osvaldo,A).
   avo(osvaldo,joana)
   avo(osvaldo,paulo)
   avo(osvaldo,bia)
--- irmao, tio e primo
?- irmao(jake,A).
   irmao(jake,ana)
?- tio(A,jake).
   tio(paulo,jake)
?- primo(leo,A).
   primo(leo,jake)
   primo(leo,ana)
--- ancestral (recursiva)
?- ancestral(osvaldo,A).
   ancestral(osvaldo,vern)
   ancestral(osvaldo,tereza)
   ancestral(osvaldo,joana)
   ancestral(osvaldo,paulo)
   ancestral(osvaldo,jake)
   ancestral(osvaldo,ana)
   ancestral(osvaldo,leo)
   ancestral(osvaldo,bia)
?- ancestral(A,jake).
   ancestral(joana,jake)
   ancestral(jorge,jake)
   ancestral(lucia,jake)
   ancestral(marta,jake)
   ancestral(osvaldo,jake)
   ancestral(vern,jake)
?- ancestral(jake,A).
   false.
?- irmao(A,B).
   irmao(vern,tereza)
   irmao(tereza,vern)
   irmao(joana,paulo)
   irmao(paulo,joana)
   irmao(jake,ana)
   irmao(ana,jake)
?- tio(A,B).
   tio(vern,bia)
   tio(paulo,jake)
   tio(paulo,ana)
?- ancestral(A,leo).
   ancestral(dora,leo)
   ancestral(paulo,leo)
   ancestral(lucia,leo)
   ancestral(marta,leo)
   ancestral(osvaldo,leo)
   ancestral(vern,leo)
?- primo(A,B).
   primo(bia,joana)
   primo(bia,paulo)
   primo(jake,leo)
   primo(ana,leo)
   primo(joana,bia)
   primo(paulo,bia)
   primo(leo,jake)
   primo(leo,ana)
[vern,tereza,joana,paulo,jake,ana,leo,bia]
```


Ready
Light
Wiki




```
% Aula 13 - Etapa 2: regras e recursao (Sebesta, sec. 16.6)
% Uma REGRA tem a forma  cabeca :- corpo.   Le-se: "a cabeca e verdadeira SE o corpo for verda
% A virgula no corpo significa E. Variaveis comecam com letra MAIUSCULA.
% Duas regras com a mesma cabeca significam OU: vale qualquer uma delas.

% ----- fatos: mae(Mae, Filho) e pai(Pai, Filho)
mae(lucia, vern).
mae(lucia, tereza).
mae(marta, joana).
mae(marta, paulo).
mae(tereza, bia).
mae(joana, jake).
mae(joana, ana).
mae(dora, leo).

pai(osvaldo, vern).
pai(osvaldo, tereza).
pai(vern, joana).
pai(vern, paulo).
pai(carlos, bia).
pai(jorge, jake).
pai(jorge, ana).
pai(paulo, leo).

% ----- fatos: homem(X) e mulher(X)
homem(osvaldo).
homem(vern).
homem(carlos).
homem(paulo).
homem(jorge).
homem(jake).
homem(leo).

mulher(lucia).
mulher(marta).
mulher(tereza).
mulher(joana).
mulher(bia).
mulher(ana).
mulher(dora).

% ----- regras
progenitor(X, Y) :- mae(X, Y).          % X e progenitor de Y se X e mae de Y
progenitor(X, Y) :- pai(X, Y).          % ... OU se X e pai de Y

avo(X, Z) :- progenitor(X, Y), progenitor(Y, Z).

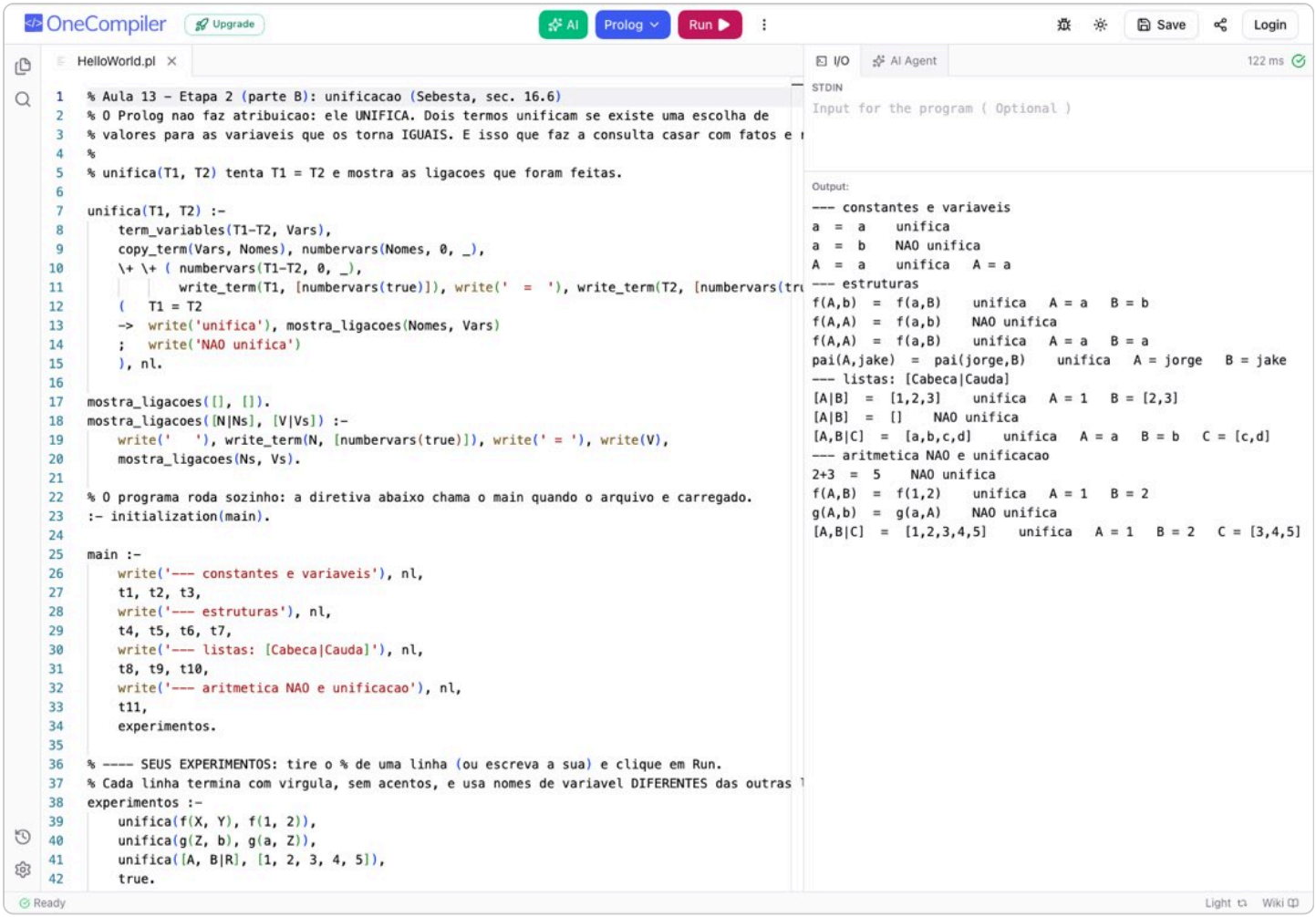
irmao(X, Y) :- mae(M, X), mae(M, Y), pai(P, X), pai(P, Y).

tio(X, Y) :- progenitor(P, Y), irmao(X, P), homem(X).

primo(X, Y) :- progenitor(P, X), progenitor(Q, Y), irmao(P, Q).

% ----- regra RECURSIVA: ancestral
ancestral(X, Y) :- progenitor(X, Y).                    % caso base
ancestral(X, Y) :- progenitor(X, Z), ancestral(Z, Y).   % caso recursivo
```

```
--- avo: duas regras progenitor encadeadas
?- avo(A,jake).
   avo(marta,jake)
   avo(vern,jake)
?- avo(osvaldo,A).
   avo(osvaldo,joana)
   avo(osvaldo,paulo)
   avo(osvaldo,bia)
--- irmao, tio e primo
?- irmao(jake,A).
   irmao(jake,jake)
   irmao(jake,ana)
?- tio(A,jake).
   tio(paulo,jake)
?- primo(leo,A).
   primo(leo,jake)
   primo(leo,ana)
   primo(leo,leo)
--- ancestral (recursiva)
?- ancestral(osvaldo,A).
   ancestral(osvaldo,vern)
   ancestral(osvaldo,tereza)
   ancestral(osvaldo,joana)
   ancestral(osvaldo,paulo)
   ancestral(osvaldo,jake)
   ancestral(osvaldo,ana)
   ancestral(osvaldo,leo)
   ancestral(osvaldo,bia)
?- ancestral(A,jake).
   ancestral(joana,jake)
   ancestral(jorge,jake)
   ancestral(lucia,jake)
   ancestral(marta,jake)
   ancestral(osvaldo,jake)
   ancestral(vern,jake)
?- ancestral(jake,A).
   false.
```




```prolog
% Aula 13 - Etapa 2 (parte B): unificacao (Sebesta, sec. 16.6)
% O Prolog nao faz atribuicao: ele UNIFICA. Dois termos unificam se existe uma escolha de
% valores para as variaveis que os torna IGUAIS. E isso que faz a consulta casar com fatos e r
%
% unifica(T1, T2) tenta T1 = T2 e mostra as ligacoes que foram feitas.

unifica(T1, T2) :-
    term_variables(T1-T2, Vars),
    copy_term(Vars, Nomes), numbervars(Nomes, 0, _),
    \+ \+ ( numbervars(T1-T2, 0, _),
        write_term(T1, [numbervars(true)]), write('  =  '), write_term(T2, [numbervars(tru
    (   T1 = T2
    ->  write('unifica'), mostra_ligacoes(Nomes, Vars)
    ;   write('NAO unifica')
    ), nl.

mostra_ligacoes([], []).
mostra_ligacoes([N|Ns], [V|Vs]) :-
    write('   '), write_term(N, [numbervars(true)]), write(' = '), write(V),
    mostra_ligacoes(Ns, Vs).

% O programa roda sozinho: a diretiva abaixo chama o main quando o arquivo e carregado.
:- initialization(main).

main :-
    write('--- constantes e variaveis'), nl,
    t1, t2, t3,
    write('--- estruturas'), nl,
    t4, t5, t6, t7,
    write('--- listas: [Cabeca|Cauda]'), nl,
    t8, t9, t10,
    write('--- aritmetica NAO e unificacao'), nl,
    t11,
    experimentos.

% ---- SEUS EXPERIMENTOS: tire o % de uma linha (ou escreva a sua) e clique em Run.
% Cada linha termina com virgula, sem acentos, e usa nomes de variavel DIFERENTES das outras
experimentos :-
    unifica(f(X, Y), f(1, 2)),
    unifica(g(Z, b), g(a, Z)),
    unifica([A, B|R], [1, 2, 3, 4, 5]),
    true.
```

2.1 Por que ancestral precisa de duas cláusulas? Qual é o caso base e qual é o caso recursivo?

Porque a definição tem dois casos, que funcionam como um OU. Caso base: ancestral(X, Y) :- progenitor(X, Y), X é ancestral de Y se é pai ou mãe dele (para a recursão). Caso recursivo: ancestral(X, Y) :- progenitor(X, Z), ancestral(Z, Y), X é ancestral de Y se é progenitor de algum Z que é ancestral de Y. Sem o caso base a recursão nunca terminaria com sucesso; sem o recursivo só acharia pais e mães, não avós, bisavós etc.

2.2 Apague X \== Y da regra irmao e consulte irmao(jake, X). O que mudou? Por que a meta é necessária?

Sem X \== Y, irmao(jake, X) passou a responder irmao(jake, jake) e irmao(jake, ana): jake virou irmão de si mesmo. Isso acontece porque jake tem a mesma mãe e o mesmo pai que ele mesmo, então todas as outras metas são satisfeitas com X = Y. O efeito se espalha: primo(leo, X) também passou a dar primo(leo, leo). A meta X \== Y é necessária para excluir a própria pessoa.

2.3 Por que f(X,X)=f(a,b) não unifica mas f(X,X)=f(a,Y) unifica? Quais valores ficam em X e Y?

Em f(X, X) = f(a, b), o mesmo X teria que unificar com a e com b ao mesmo tempo; como a e b são constantes diferentes, não há valor possível e falha. Em f(X, X) = f(a, Y), o primeiro argumento liga X = a, e o segundo exige X = Y, então Y também fica ligado a a. Resultado: X = a e Y = a.

2.4 avo responde em várias direções. Isso é possível numa função de Java ou Scheme? Explique a diferença.

Não diretamente. Em Java ou Scheme uma função tem entradas e uma saída fixas: avo(neto) devolveria os avós, e para achar os netos de alguém seria preciso escrever outra função. No Prolog, avo(X, Z) é uma relação, não uma função: não existe argumento de entrada ou de saída, qualquer um pode ser variável, e a unificação + backtracking procuram os valores que tornam a relação verdadeira. Por isso a mesma regra serve para avo(X, jake) e para avo(osvaldo, X).

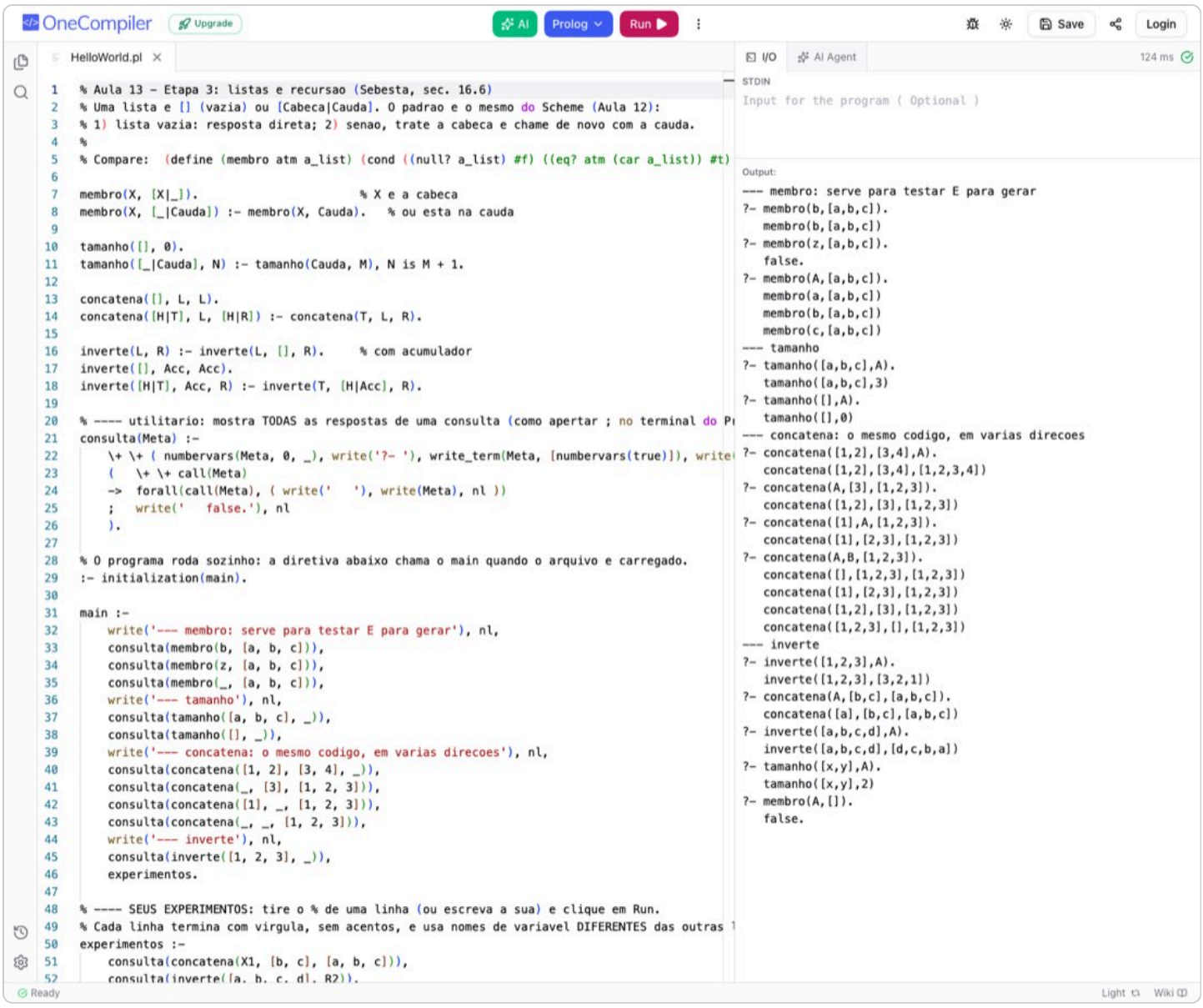
Etapa 3: Listas: o mesmo padrão do Scheme, em formato de relação

Previsão: O que concatena(X, Y, [1,2,3]) vai responder? Quantas respostas?

Deve dar todas as formas de dividir [1, 2, 3] em duas partes: [] e [1,2,3], [1] e [2,3], [1,2] e [3], [1,2,3] e []. São 4 respostas.

Saída do listas.pl

```
Saída no print do OneCompiler em anexo (código + saída).
```



3.1 Em tamanho, qual é o caso base e qual é o passo recursivo? O que fica pendente a cada chamada?

Caso base: tamanho([], 0), a lista vazia tem tamanho 0. Passo recursivo: tamanho([_|Cauda], N) :- tamanho(Cauda, M), N is M + 1, o tamanho é o da cauda mais 1. Em tamanho([a, b, c], N), cada chamada fica esperando a volta para fazer o "N is M + 1": tamanho([b,c]) fica pendente com +1, tamanho([c]) com +1, até tamanho([]) = 0; na volta calcula 1, 2 e 3. Ou seja, fica pendente a soma, como na recursão comum (não em cauda) da Aula 12.

3.2 Por que concatena(X,Y,[1,2,3]) tem 4 respostas? Compare com anexar do Scheme: o que ele não consegue fazer?

Porque concatena é uma relação e as duas primeiras listas estão livres: o Prolog procura todos os pares X, Y cuja concatenação é [1, 2, 3]. A primeira cláusula dá X = [] e, no backtracking, a segunda tira um elemento de cada vez

para X, gerando as 4 divisões possíveis (com 0, 1, 2 e 3 elementos em X). O anexar do Scheme é uma função: recebe as duas listas e só devolve a junção. Ele não consegue separar uma lista, descobrir o prefixo que falta ou listar todas as divisões; para isso seria preciso escrever outras funções.

3.3 membro(X,[a,b,c]) gerou a, b e c sem laço. Que mecanismo do Prolog fez isso?

O backtracking junto com a unificação. A primeira cláusula membro(X, [X|_]) unifica X com a cabeça (a). Quando o Prolog pede outra resposta, ele volta ao ponto de escolha e tenta a segunda cláusula, que chama membro com a cauda, e assim aparecem b e depois c. A repetição vem da recursão e da busca por alternativas, não de um laço.

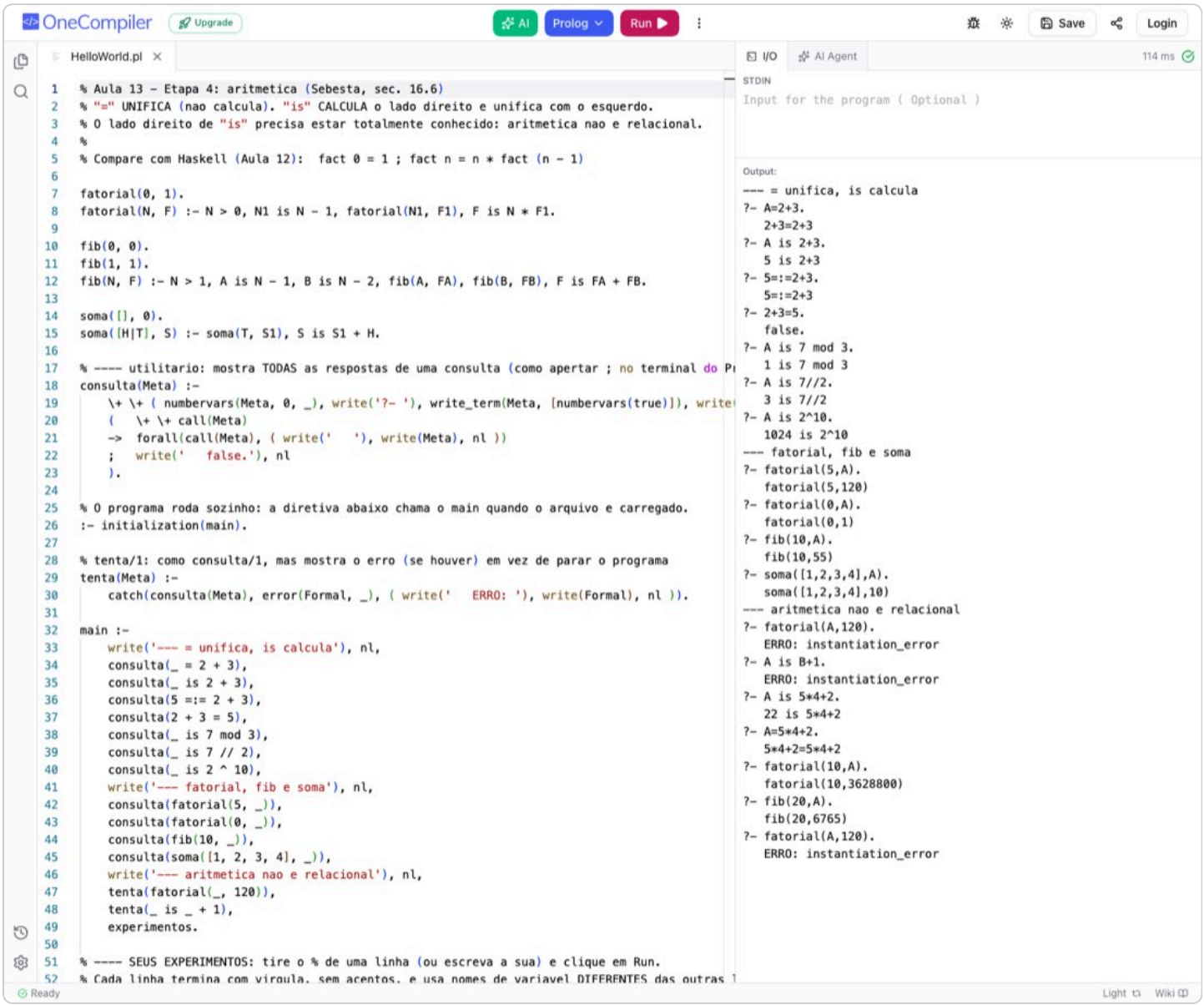
Etapa 4: Aritmética: onde o Prolog deixa de ser relacional

Previsão: O que mostram X = 2 + 3 e X is 2 + 3? E fatorial(N, 120)?

X = 2 + 3 mostra X = 2+3 (só unifica a estrutura, não calcula). X is 2 + 3 mostra 5. fatorial(N, 120) deve dar erro, porque N > 0 e N1 is N - 1 recebem N sem valor (erro de instanciação).

Saída do aritmetica.pl

```
Saída no print do OneCompiler em anexo (código + saída).
```



4.1 Explique com exemplos a diferença entre =, is e =:=.

= é unificação: só casa estruturas, não calcula. X = 2 + 3 deixa X valendo a estrutura 2+3, e 2 + 3 = 5 dá false.
is calcula o lado direito e unifica o resultado com o esquerdo: X is 2 + 3 dá X = 5; X is 5 * 4 + 2 dá 22. O lado direito precisa estar todo conhecido.
=:= compara os valores numéricos das duas expressões (avalia os dois lados): 5 =:= 2 + 3 é verdadeiro, mas não liga variáveis.

4.2 Por que fatorial(N,120) dá erro e concatena(X,[3],[1,2,3]) funcionou? O que há de diferente na aritmética?

Em fatorial(N, 120), N está livre, e a primeira coisa da regra é N > 0 e N1 is N - 1: as operações aritméticas exigem que os valores estejam conhecidos, então dá instantiation_error. A aritmética do Prolog é avaliada como uma função de mão única (entrada -> resultado) e não pode ser "invertida". Já concatena usa só unificação sobre listas, que funciona em qualquer direção, então concatena(X, [3], [1, 2, 3]) consegue descobrir X = [1, 2].

4.3 Compare o fatorial em Prolog com o fact em Haskell: o que é igual e o que é diferente?

Igual: a estrutura é a mesma, um caso base (fatorial de 0 é 1) e um caso recursivo (N * fatorial(N - 1)), sem laço e sem variável que muda.
Diferente: o Haskell define uma função que devolve o valor (fact n = ...), enquanto no Prolog é uma relação e o resultado entra como argumento extra (fatorial(N, F)). No Prolog é preciso usar is para calcular e N1 é uma variável nova, e é preciso o teste N > 0 explícito, já que não há casamento de padrões que escolha só uma cláusula; o Haskell escolhe o caso pelo padrão. Além disso, o Haskell não volta atrás em alternativas, o Prolog sim.

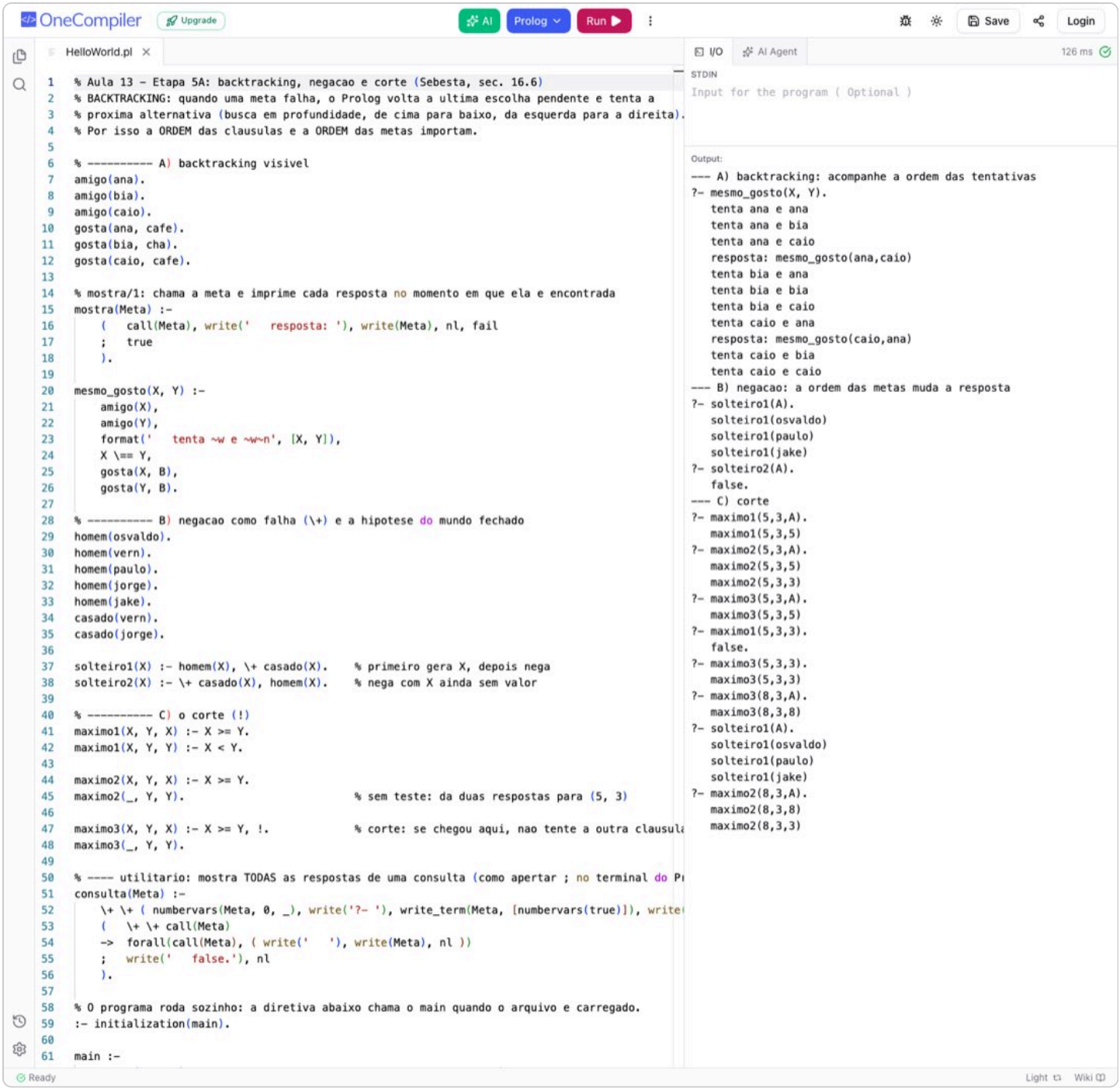
Etapa 5: Controlando (e sofrendo com) a busca

Previsão: qual de solteiro1/solteiro2 dá false? maximo2(5,3,M) uma ou duas respostas? maximo3(5,3,3) true ou false?

solteiro2(X) deve dar false, porque \+ casado(X) com X livre é falso (existe alguém casado). maximo2(5, 3, M) deve dar duas respostas (5 e 3, porque a segunda cláusula não tem teste). maximo3(5, 3, 3) deve dar true, mesmo sendo errado, porque a primeira cláusula não unifica e o corte nem é executado.

Saída de controle.pl e mensagem de ancestral_ruim

```
Saída no print do OneCompiler em anexo (código + saída).
```




```
% Aula 13 - Etapa 5A: backtracking, negacao e corte (Sebesta, sec. 16.6)
% BACKTRACKING: quando uma meta falha, o Prolog volta a ultima escolha pendente e tenta a
% proxima alternativa (busca em profundidade, de cima para baixo, da esquerda para a direita).
% Por isso a ORDEM das clausulas e a ORDEM das metas importam.

% ---------- A) backtracking visivel
amigo(ana).
amigo(bia).
amigo(caio).
gosta(ana, cafe).
gosta(bia, cha).
gosta(caio, cafe).

% mostra/1: chama a meta e imprime cada resposta no momento em que ela e encontrada
mostra(Meta) :-
    (   call(Meta), write('   resposta: '), write(Meta), nl, fail
    ;   true
    ).

mesmo_gosto(X, Y) :-
    amigo(X),
    amigo(Y),
    format('   tenta ~w e ~w~n', [X, Y]),
    X \== Y,
    gosta(X, B),
    gosta(Y, B).

% ---------- B) negacao como falha (\+) e a hipotese do mundo fechado
homem(osvaldo).
homem(vern).
homem(paulo).
homem(jorge).
homem(jake).
casado(vern).
casado(jorge).

solteiro1(X) :- homem(X), \+ casado(X).    % primeiro gera X, depois nega
solteiro2(X) :- \+ casado(X), homem(X).    % nega com X ainda sem valor

% ---------- C) o corte (!)
maximo1(X, Y, X) :- X >= Y.
maximo1(X, Y, Y) :- X < Y.

maximo2(X, Y, X) :- X >= Y.
maximo2(_, Y, Y).                          % sem teste: da duas respostas para (5, 3)

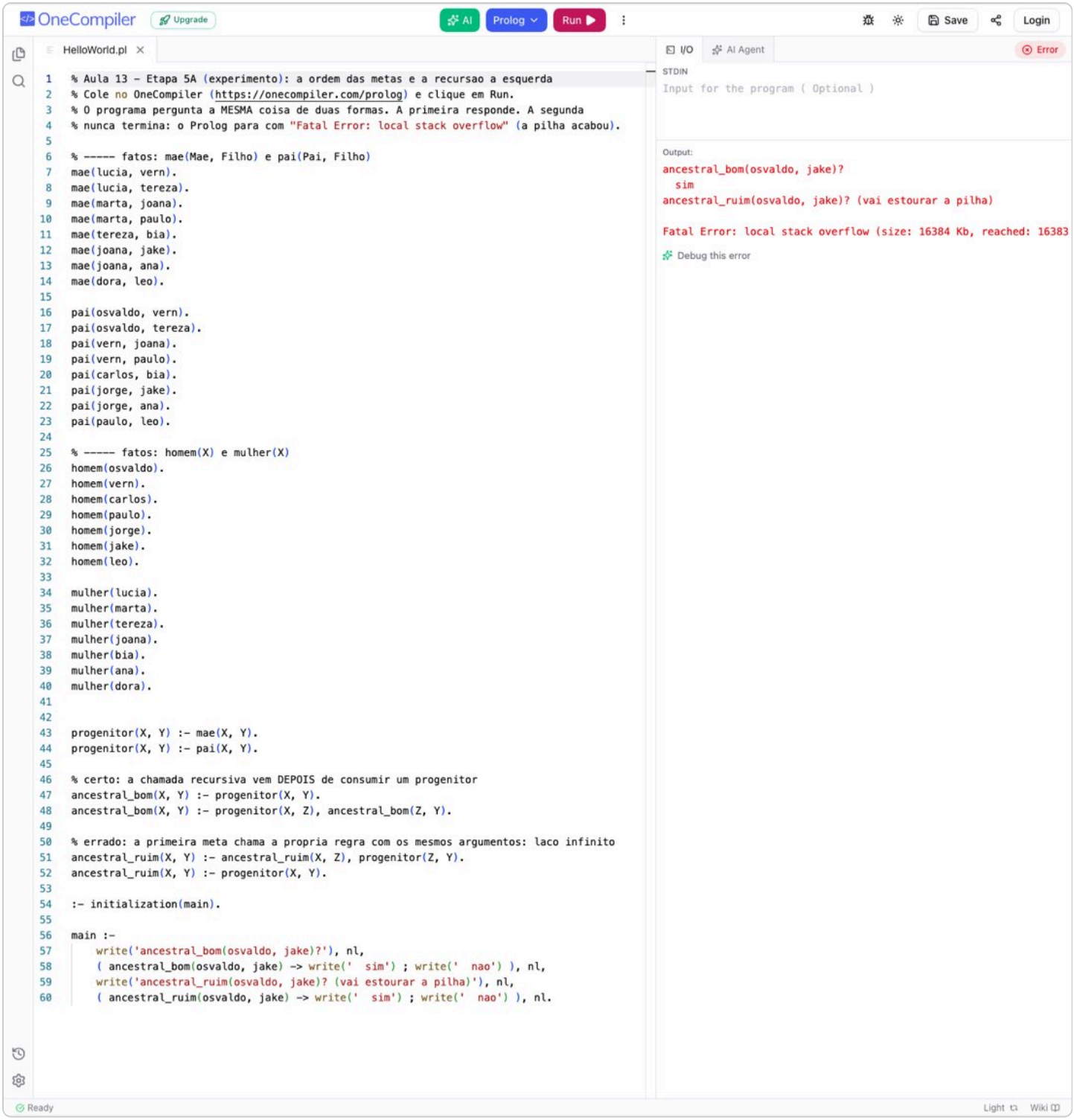
maximo3(X, Y, X) :- X >= Y, !.             % corte: se chegou aqui, nao tente a outra clausula
maximo3(_, Y, Y).

% ---- utilitario: mostra TODAS as respostas de uma consulta (como apertar ; no terminal do Pr
consulta(Meta) :-
    \+ \+ ( numbervars(Meta, 0, _), write('?- '), write_term(Meta, [numbervars(true)]), write(
    (   \+ \+ call(Meta)
    ->  forall(call(Meta), ( write('   '), write(Meta), nl ))
    ;   write('   false.'), nl
    ).

% O programa roda sozinho: a diretiva abaixo chama o main quando o arquivo e carregado.
:- initialization(main).

main :-
```

```
--- A) backtracking: acompanhe a ordem das tentativas
?- mesmo_gosto(X, Y).
   tenta ana e ana
   tenta ana e bia
   tenta ana e caio
   resposta: mesmo_gosto(ana,caio)
   tenta bia e ana
   tenta bia e bia
   tenta bia e caio
   tenta caio e ana
   resposta: mesmo_gosto(caio,ana)
   tenta caio e bia
   tenta caio e caio
--- B) negacao: a ordem das metas muda a resposta
?- solteiro1(A).
   solteiro1(osvaldo)
   solteiro1(paulo)
   solteiro1(jake)
?- solteiro2(A).
   false.
--- C) corte
?- maximo1(5,3,A).
   maximo1(5,3,5)
?- maximo2(5,3,A).
   maximo2(5,3,5)
   maximo2(5,3,3)
?- maximo3(5,3,A).
   maximo3(5,3,5)
?- maximo1(5,3,3).
   false.
?- maximo3(5,3,3).
   maximo3(5,3,3)
?- maximo3(8,3,A).
   maximo3(8,3,8)
?- solteiro1(A).
   solteiro1(osvaldo)
   solteiro1(paulo)
   solteiro1(jake)
?- maximo2(8,3,A).
   maximo2(8,3,8)
   maximo2(8,3,3)
```




```
% Aula 13 - Etapa 5A (experimento): a ordem das metas e a recursao a esquerda
% Cole no OneCompiler (https://onecompiler.com/prolog) e clique em Run.
% O programa pergunta a MESMA coisa de duas formas. A primeira responde. A segunda
% nunca termina: o Prolog para com "Fatal Error: local stack overflow" (a pilha acabou).

% ----- fatos: mae(Mae, Filho) e pai(Pai, Filho)
mae(lucia, vern).
mae(lucia, tereza).
mae(marta, joana).
mae(marta, paulo).
mae(tereza, bia).
mae(joana, jake).
mae(joana, ana).
mae(dora, leo).

pai(osvaldo, vern).
pai(osvaldo, tereza).
pai(vern, joana).
pai(vern, paulo).
pai(carlos, bia).
pai(jorge, jake).
pai(jorge, ana).
pai(paulo, leo).

% ----- fatos: homem(X) e mulher(X)
homem(osvaldo).
homem(vern).
homem(carlos).
homem(paulo).
homem(jorge).
homem(jake).
homem(leo).

mulher(lucia).
mulher(marta).
mulher(tereza).
mulher(joana).
mulher(bia).
mulher(ana).
mulher(dora).


progenitor(X, Y) :- mae(X, Y).
progenitor(X, Y) :- pai(X, Y).

% certo: a chamada recursiva vem DEPOIS de consumir um progenitor
ancestral_bom(X, Y) :- progenitor(X, Y).
ancestral_bom(X, Y) :- progenitor(X, Z), ancestral_bom(Z, Y).

% errado: a primeira meta chama a propria regra com os mesmos argumentos: laco infinito
ancestral_ruim(X, Y) :- ancestral_ruim(X, Z), progenitor(Z, Y).
ancestral_ruim(X, Y) :- progenitor(X, Y).

:- initialization(main).

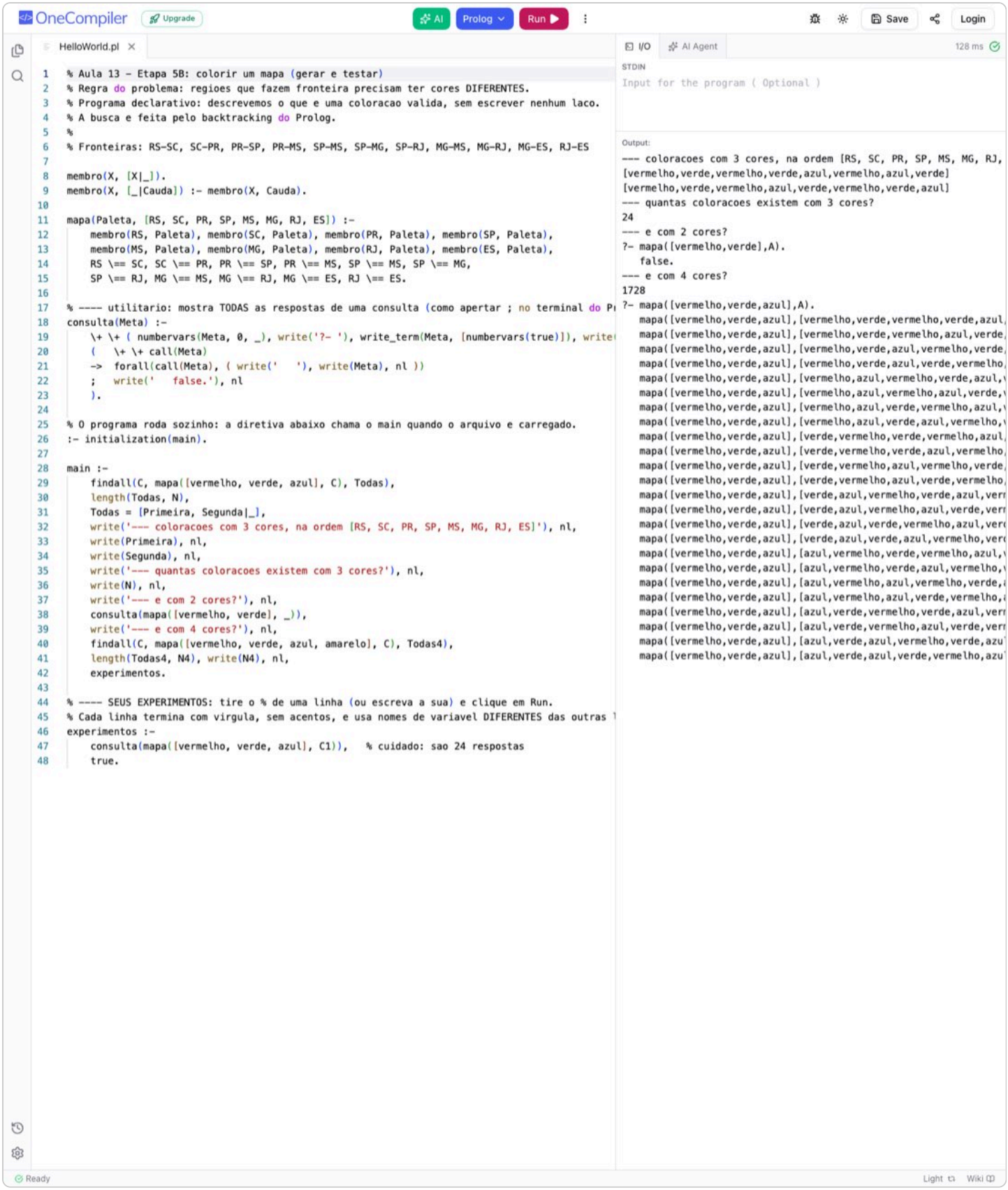
main :-
    write('ancestral_bom(osvaldo, jake)?'), nl,
    ( ancestral_bom(osvaldo, jake) -> write('  sim') ; write('  nao') ), nl,
    write('ancestral_ruim(osvaldo, jake)? (vai estourar a pilha)'), nl,
    ( ancestral_ruim(osvaldo, jake) -> write('  sim') ; write('  nao') ), nl.
```

Previsão: Quantas colorações existem com 3 cores? E com 2 cores?

Com 3 cores imagino algumas dezenas (talvez 24). Com 2 cores, nenhuma: PR, SP e MS fazem fronteira entre si (formam um triângulo), então precisam de 3 cores diferentes.

Saída do mapa.pl

```
Saída no print do OneCompiler em anexo (código + saída).
```




```
% Aula 13 - Etapa 5B: colorir um mapa (gerar e testar)
% Regra do problema: regioes que fazem fronteira precisam ter cores DIFERENTES.
% Programa declarativo: descrevemos o que e uma coloracao valida, sem escrever nenhum laco.
% A busca e feita pelo backtracking do Prolog.
%
% Fronteiras: RS-SC, SC-PR, PR-SP, PR-MS, SP-MS, SP-MG, SP-RJ, MG-MS, MG-RJ, MG-ES, RJ-ES

membro(X, [X|_]).
membro(X, [_|Cauda]) :- membro(X, Cauda).

mapa(Paleta, [RS, SC, PR, SP, MS, MG, RJ, ES]) :-
    membro(RS, Paleta), membro(SC, Paleta), membro(PR, Paleta), membro(SP, Paleta),
    membro(MS, Paleta), membro(MG, Paleta), membro(RJ, Paleta), membro(ES, Paleta),
    RS \== SC, SC \== PR, PR \== SP, PR \== MS, SP \== MS, SP \== MG,
    SP \== RJ, MG \== MS, MG \== RJ, MG \== ES, RJ \== ES.

% ---- utilitario: mostra TODAS as respostas de uma consulta (como apertar ; no terminal do Pr
consulta(Meta) :-
    \+ \+ ( numbervars(Meta, 0, _), write('?- '), write_term(Meta, [numbervars(true)]), write(
    (   \+ \+ call(Meta)
    ->  forall(call(Meta), ( write('   '), write(Meta), nl ))
    ;   write('   false.'), nl
    ).

% O programa roda sozinho: a diretiva abaixo chama o main quando o arquivo e carregado.
:- initialization(main).

main :-
    findall(C, mapa([vermelho, verde, azul], C), Todas),
    length(Todas, N),
    Todas = [Primeira, Segunda|_],
    write('--- coloracoes com 3 cores, na ordem [RS, SC, PR, SP, MS, MG, RJ, ES]'), nl,
    write(Primeira), nl,
    write(Segunda), nl,
    write('--- quantas coloracoes existem com 3 cores?'), nl,
    write(N), nl,
    write('--- e com 2 cores?'), nl,
    consulta(mapa([vermelho, verde], _)),
    write('--- e com 4 cores?'), nl,
    findall(C, mapa([vermelho, verde, azul, amarelo], C), Todas4),
    length(Todas4, N4), write(N4), nl,
    experimentos.

% ---- SEUS EXPERIMENTOS: tire o % de uma linha (ou escreva a sua) e clique em Run.
% Cada linha termina com virgula, sem acentos, e usa nomes de variavel DIFERENTES das outras
experimentos :-
    consulta(mapa([vermelho, verde, azul], C1)),   % cuidado: sao 24 respostas
    true.
```

5.1 Com o rastro do bloco A, explique como o Prolog encontrou as respostas de mesmo_gosto(X,Y). Que meta falhou e para onde voltou?

O Prolog escolhe X = ana (primeiro amigo) e Y = ana; em "tenta ana e ana" a meta que falha é X \== Y. Então ele volta ao último ponto de escolha, que é amigo(Y), e tenta Y = bia: X \== Y passa, gosta(ana, B) dá B = cafe, mas gosta(bia, cafe) falha e ele volta de novo para Y = caio, onde gosta(caio, cafe) dá certo: primeira resposta mesmo_gosto(ana, caio). Quando acabam os Y, volta para amigo(X) e repete com X = bia (nenhum par dá certo, bia gosta de cha) e X = caio, encontrando mesmo_gosto(caio, ana). Por fim testa caio com bia e caio e termina.

5.2 Por que solteiro2 dá false e solteiro1 não? Por que solteiro1 afirma que osvaldo é solteiro?

Em solteiro2, \+ casado(X) é executado com X ainda sem valor: casado(X) consegue ser provado (X = vern), então \+ casado(X) falha e a regra toda dá false. Em solteiro1, homem(X) gera primeiro um valor (osvaldo, vern...) e só depois a negação testa aquela pessoa específica, então funciona. Osvaldo aparece como solteiro por causa da negação como falha e do mundo fechado: não existe o fato casado(osvaldo), então o Prolog não consegue provar que ele é casado e conclui que é solteiro, mesmo ele sendo pai de vern (e provavelmente casado com lucia na vida real).

5.3 maximo3(5,3,3) deu true. Explique por que o corte não protegeu esse uso. Como você corrigiria?

Em maximo3(5, 3, 3), o terceiro argumento já vem como 3, então a cabeça da primeira cláusula maximo3(X, Y, X) não unifica (X teria que ser 5 e 3). Ela falha antes de chegar no X >= Y e no corte, então o corte nunca é executado e o Prolog tenta a segunda cláusula maximo3(_, Y, Y), que não tem teste e aceita 3. Correção: deixar a cabeça com uma variável livre e fazer a unificação depois do corte:

```
maximo3(X, Y, M) :- X >= Y, !, M = X.
maximo3(_, Y, Y).
```

Assim maximo3(5, 3, 3) passa pelo corte e falha em 3 = 5, dando false.

5.4 Por que ancestral_ruim estoura a pilha e ancestral_bom não, se dizem a mesma coisa logicamente?

Porque o Prolog resolve as metas da esquerda para a direita e as cláusulas de cima para baixo. Em ancestral_ruim, a primeira cláusula começa chamando ancestral_ruim(X, Z) de novo, sem ter consumido nenhum fato: é uma recursão à esquerda, que chama a si mesma infinitamente com a mesma meta e nunca chega no progenitor, até estourar a pilha ("Fatal Error: local stack overflow"). Em ancestral_bom, o caso base vem primeiro e o recursivo primeiro consome um progenitor(X, Z), aproximando-se da resposta a cada chamada, então termina. Logicamente são equivalentes, mas a ordem de resolução do Prolog não é (deficiência citada no Sebesta, §16.7).

5.5 Quantas colorações com 3, 2 e 4 cores? Por que o programa é declarativo apesar de fazer busca?

Encontrou 24 colorações com 3 cores, nenhuma com 2 cores (false) e 1728 com 4 cores. O programa é declarativo porque só descreve o que é uma solução válida: cada estado recebe uma cor da paleta e estados vizinhos têm cores diferentes. Não há laço, nem algoritmo de busca, nem ordem de tentativas escrita por mim; quem faz a busca (gerar e testar) é o backtracking do próprio Prolog.

Fechamento: O que você viu e onde isso se encaixa

6.1 Explique a diferença entre o paradigma lógico e o funcional usando concatena e anexar.

No funcional, anexar(l1, l2) é uma função: recebe duas listas e devolve a junção, sempre na mesma direção (entradas -> saída). No lógico, concatena(A, B, C) é uma relação entre três listas: a mesma definição junta, separa (concatena(X, Y, [1,2,3])) e descobre partes que faltam, porque qualquer argumento pode ser variável e o Prolog procura os valores por unificação e backtracking.

6.2 Cite duas deficiências do Prolog que você observou, ligando cada uma a um programa.

1) Controle da ordem de resolução: ancestral_ruim e ancestral_bom dizem a mesma coisa logicamente, mas a ruim estoura a pilha (laco_infinito.pl); o programador precisa pensar em como o Prolog executa. O mesmo vale para o corte em maximo3 (controle.pl), que mudou o significado do programa e deu true para maximo3(5, 3, 3).
2) Mundo fechado e negação como falha: solteiro1 afirma que osvaldo é solteiro só porque não há o fato casado(osvaldo), e solteiro2 dá false só por mudar a ordem das metas (controle.pl). Também a aritmética não relacional: fatorial(N, 120) dá instantiation_error (aritmetica.pl).

6.3 Para que tipo de problema você escolheria Prolog em vez de Java? E para qual não escolheria?

Escolheria Prolog para problemas baseados em regras, relações e busca: sistemas especialistas (regras se-então), consultas dedutivas sobre bases de conhecimento (como família, permissões, dependências), problemas de restrições e combinatória (colorir mapa, SEND+MORE, escalas, horários) e análise de linguagem. Nesses casos descrever o problema é mais simples que escrever o algoritmo de busca. Não escolheria para sistemas com muita entrada/saída, interface, estado mutável e regras de negócio sequenciais (um sistema web, um app, um sistema de pedidos), nem para processamento numérico pesado, onde Java é mais rápido, previsível e tem muito mais bibliotecas e ferramentas.

Exercícios opcionais

Opcionais: resultado dos testes

```
Saída no print do OneCompiler em anexo (código + saída).
```

OneCompiler Upgrade AI Prolog Run Save Login

HelloWorld.pl

```
% Aula 13 - Exercicios opcionais (enunciado)
% Complete cada exercicio logo abaixo do seu titulo e execute: o programa mostra "ok" ou "FALH
% OneCompiler (https://onecompiler.com/prolog): cole o arquivo inteiro e clique em Run.
% Observacao: o OneCompiler nao aceita letras acentuadas no codigo: escreva sem acentos.

% ----- fatos: mae(Mae, Filho) e pai(Pai, Filho)
mae(lucia, vern).
mae(lucia, tereza).
mae(marta, joana).
mae(marta, paulo).
mae(tereza, bia).
mae(joana, jake).
mae(joana, ana).
mae(dora, leo).

pai(osvaldo, vern).
pai(osvaldo, tereza).
pai(vern, joana).
pai(vern, paulo).
pai(carlos, bia).
pai(jorge, jake).
pai(jorge, ana).
pai(paulo, leo).

% ----- fatos: homem(X) e mulher(X)
homem(osvaldo).
homem(vern).
homem(carlos).
homem(paulo).
homem(jorge).
homem(jake).
homem(leo).

mulher(lucia).
mulher(marta).
mulher(tereza).
mulher(joana).
mulher(bia).
mulher(ana).
mulher(dora).


% ----- ja prontos (use nos exercicios)
progenitor(X, Y) :- mae(X, Y).
progenitor(X, Y) :- pai(X, Y).
irmao(X, Y) :- mae(M, X), mae(M, Y), pai(P, X), pai(P, Y), X \== Y.

membro(X, [X|_]).
membro(X, [_|Cauda]) :- membro(X, Cauda).

soma([], 0).
soma([H|T], S) :- soma(T, S1), S is S1 + H.

% grafo de cidades (exercicio 5): ligacoes de ida, com ciclos
liga(curitiba, ponta_grossa).
liga(ponta_grossa, guarapuava).
liga(guarapuava, cascavel).
liga(cascavel, foz).
liga(curitiba, joinville).
liga(joinville, florianopolis).
liga(florianopolis, curitiba).
liga(guarapuava, ponta_grossa).

% tabela de funcionarios (exercicio 7): funcionario(Nome, Departamento, Salario)
funcionario(ana, ti, 7000).
funcionario(bruno, ti, 5500).
funcionario(carla, rh, 4800).
funcionario(daniel, rh, 4200).
funcionario(elisa, vendas, 3900).

% === EXERCICIO 1: irma/2 e tia/2 (use progenitor/2, irmao/2 e mulher/1) ===
% irma(X, Y):  X e irma de Y.
% tia(X, Y):   X e tia de Y (irma de um progenitor de Y).
irma(X, Y) :- irmao(X, Y), mulher(X).
tia(X, Y) :- progenitor(P, Y), irma(X, P).

% === EXERCICIO 2: ultimo/2 ===
% ultimo(Lista, X): X e o ultimo elemento da lista.   Ex.: ultimo([a, b, c], c).
% Dica: uma lista com 1 elemento tem o proprio elemento como ultimo (caso base).
ultimo([X], X).
ultimo([_|T], X) :- ultimo(T, X).

% === EXERCICIO 3: as funcoes da Aula 12 (Scheme), agora em Prolog ===
% soma_quadrados(Lista, S):  S e a soma dos quadrados.         Ex.: [1, 2, 3] -> 14
% pares(Lista, P):           P tem so os elementos pares.        Ex.: [1, 2, 3, 4, 5, 6] -> [2
% Dica: para pares, uma clausula inclui a cabeca (H mod 2 =:= 0) e outra a descarta (H mod 2 =
soma_quadrados([], 0).
soma_quadrados([H|T], S) :- soma_quadrados(T, S1), S is S1 + H * H.

pares([], []).
pares([H|T], [H|R]) :- H mod 2 =:= 0, pares(T, R).
pares([H|T], R) :- H mod 2 =\= 0, pares(T, R).

% === EXERCICIO 4: mdc/3 e potencia/3 ===
% mdc(A, B, D): D e o maximo divisor comum (algoritmo de Euclides: mdc(A, B) = mdc(B, A mod B
```

I/O AI Agent 131 ms

STDIN

Input for the program (Optional)

Output:

```
Exercicio 1
   ok      irma(ana, jake)
   ok      irma(tereza, vern)
   ok      NAO irma(jake, ana)
   ok      tia(tereza, joana)
   ok      tia(joana, leo)
   ok      NAO tia(joana, jake)
Exercicio 2
   ok      ultimo([a,b,c], c)
   ok      ultimo([7], 7)
   ok      NAO ultimo([a,b,c], b)
Exercicio 3
   ok      soma_quadrados([1,2,3], 14)
   ok      soma_quadrados([], 0)
   ok      pares([1,2,3,4,5,6], [2,4,6])
   ok      pares([1,3], [])
Exercicio 4
   ok      mdc(12, 18, 6)
   ok      mdc(17, 5, 1)
   ok      mdc(10, 0, 10)
   ok      potencia(2, 10, 1024)
   ok      potencia(5, 0, 1)
Exercicio 5
   ok      caminho(curitiba, foz)
   ok      caminho(joinville, cascavel)
   ok      NAO caminho(foz, curitiba)
   ok      NAO caminho(curitiba, paris)
Exercicio 6
   ok      3 cores: 12 coloracoes
   ok      2 cores: 2 coloracoes
   ok      1 cor: nenhuma
Exercicio 7
   ok      colega(ana, bruno)
   ok      NAO colega(ana, carla)
   ok      NAO colega(ana, ana)
   ok      ganha_mais(ana, bruno)
   ok      NAO ganha_mais(elisa, ana)
   ok      total_depto(ti, 12500)
   ok      total_depto(rh, 9000)
```

```
% potencia(Base, Expoente, R): R = Base elevado ao Expoente, com recursao (sem usar ^).
mdc(A, 0, A).
mdc(A, B, D) :- B > 0, R is A mod B, mdc(B, R, D).

potencia(_, 0, 1).
potencia(B, E, R) :- E > 0, E1 is E - 1, potencia(B, E1, R1), R is B * R1.

% === EXERCICIO 5: caminho/2 em um grafo com ciclos ===
% caminho(A, B): existe uma sequencia de ligacoes de A ate B.
% ATENCAO: com ciclos, a versao ingenua (caminho(A,B) :- liga(A,C), caminho(C,B)) NUNCA termi
% quando B nao e alcancavel. Use uma lista de VISITADOS (use membro/2 e \+).
% Se o programa travar, interrompa e revise a sua caminho/2.
caminho(A, B) :- caminho(A, B, [A]).
caminho(A, B, _) :- liga(A, B).
caminho(A, B, Visitados) :- liga(A, C), \+ membro(C, Visitados), caminho(C, B, [C|Visitados]).

% === EXERCICIO 6: colorir um mapa menor ===
% mapa_sul(Paleta, [RS, SC, PR]): fronteiras RS-SC e SC-PR; regioes vizinhas tem cores differ
% Use membro/2 para escolher as cores e \== para comparar. Com 3 cores devem existir 12 colora
mapa_sul(Paleta, [RS, SC, PR]) :-
    membro(RS, Paleta), membro(SC, Paleta), membro(PR, Paleta),
    RS \== SC, SC \== PR.
```

Ready Light Wiki

OneCompiler Upgrade AI Prolog Run Save Login

HelloWorld.pl

```
% Aula 13 - Autopesquisa: programacao logica com restricoes (solver FD do GNU Prolog)
% Cole no OneCompiler (https://onecompiler.com/prolog) e clique em Run.
%
% Enigma classico: cada letra e um digito diferente (0 a 9), sem zero a esquerda, e
%       S E N D
%     + M O R E
%     ---------
%     M O N E Y
%
% Em vez de "gerar e testar" (como no mapa), declaramos RESTRICOES e o resolvedor poda a busca
% (O GNU Prolog usa fd_domain, fd_all_different, #= e fd_labeling.)

:- initialization(main).

puzzle([S, E, N, D, M, O, R, Y]) :-
    fd_domain([S, E, N, D, M, O, R, Y], 0, 9),
    fd_all_different([S, E, N, D, M, O, R, Y]),
    S #\= 0,
    M #\= 0,
    1000*S + 100*E + 10*N + D + 1000*M + 100*O + 10*R + E #=
        10000*M + 1000*O + 100*N + 10*E + Y,
    fd_labeling([S, E, N, D, M, O, R, Y]).

main :-
    puzzle([S, E, N, D, M, O, R, Y]),
    format('  ~w~w~w~w~n', [S, E, N, D]),
    format('+ ~w~w~w~w~n', [M, O, R, E]),
    format('= ~w~w~w~w~w~n', [M, O, N, E, Y]).
```

I/O AI Agent 124 ms

STDIN

Input for the program (Optional)

Output:

```
  9567
+ 1085
= 10652
```

Ready Light Wiki

E1 Regras de família: código escrito

```
irma(X, Y) :- irmao(X, Y), mulher(X).
tia(X, Y) :- progenitor(P, Y), irma(X, P).
```

E2 Listas: código escrito

```
ultimo([X], X).
ultimo([_|T], X) :- ultimo(T, X).
```

E3 As funções da Aula 12: código escrito

```
soma_quadrados([], 0).
soma_quadrados([H|T], S) :- soma_quadrados(T, S1), S is S1 + H * H.

pares([], []).
pares([H|T], [H|R]) :- H mod 2 =:= 0, pares(T, R).
pares([H|T], R) :- H mod 2 =\= 0, pares(T, R).
```

E4 Aritmética: código escrito

```
mdc(A, 0, A).
mdc(A, B, D) :- B > 0, R is A mod B, mdc(B, R, D).

potencia(_, 0, 1).
potencia(B, E, R) :- E > 0, E1 is E - 1, potencia(B, E1, R1), R is B * R1.
```

E5 Grafo com ciclos: código escrito

```
caminho(A, B) :- caminho(A, B, [A]).
caminho(A, B, _) :- liga(A, B).
caminho(A, B, Visitados) :- liga(A, C), \+ membro(C, Visitados), caminho(C, B,
[C|Visitados]).
```

E6 Mapa menor: código escrito

```
mapa_sul(Paleta, [RS, SC, PR]) :-
    membro(RS, Paleta), membro(SC, Paleta), membro(PR, Paleta),
    RS \== SC, SC \== PR.
```

E7 Prolog como banco de dados: código escrito

```
colega(X, Y) :- funcionario(X, D, _), funcionario(Y, D, _), X \== Y.
ganha_mais(X, Y) :- funcionario(X, _, SX), funcionario(Y, _, SY), SX > SY.
total_depto(D, Total) :- findall(S, funcionario(_, D, S), L), soma(L, Total).
```